UNIVERSITI SAINS MALAYSIA

School of Electrical and Electronics Engineering
Universiti Sains Malaysia, Engineering Campus

SEMESTER 1, 2026/2027

EEM 441

Control & Instrumentation Laboratory

Experiment 9

PROJECT CREATION IN SCADA (IOT)

Date : _____

Group No. : _____

Group Members : _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

Experiment 9

IIoT Monitoring & Alarm Visualization for a PLC-Controlled Machine

Objectives

1. To understand the role of each layer in an IIoT monitoring pipeline: PLC, SCADA, database, and dashboard.
2. To configure communication between a Siemens S7-1200 PLC and Advantech WebAccess/SCADA.
3. To bridge SCADA tag data to a PostgreSQL database via ODBC, so that history is available outside WebAccess.
4. To design a web-based IIoT dashboard displaying real-time sensor status and visual alarms, using the WISE-IoTSuite/Dashboard application.
5. To demonstrate a complete monitoring pipeline, from a physical sensor to a live dashboard panel.

Equipment / Software Required

- MTA SkillLab i-Factory kit: conveyor sorting machine with Siemens SIMATIC S7-1200 CPU 1212C AC/DC/Rly, fitted with vibration, laser-distance, temperature/humidity, and part-counting sensors
- PC/laptop with Advantech WebAccess/SCADA V9.x installed (Project Node and SCADA Node)
- PostgreSQL, with the `psqlodbc` (x86) ODBC driver and pgAdmin4 installed
- ODBC Data Source Administrator (32-bit) — WebAccess only supports 32-bit ODBC connections
- Ethernet cable and a configured local network connection between the PC and the PLC

Introduction

Industrial IoT (IIoT) monitoring bridges field-level operational technology (OT) — sensors, actuators, and the PLC that controls them — with IT-side visualization and analytics. This experiment builds that bridge one layer at a time, using the same i-Factory conveyor rig and WebAccess/SCADA software introduced in Experiment 5.

Communication protocols used at each layer (Table 9.1): Modbus links the sensors to the PLC; TCP/IP (or OPC UA) links the PLC to WebAccess/SCADA; and ODBC links WebAccess/SCADA to the PostgreSQL database that the Dashboard ultimately reads from.

The full pipeline is shown in Figure 9.1. Each stage depends on the one before it: a tag with no PLC communication shows no data; a tag with communication but no **Log Data** setting never reaches the database; and a database with no rows has nothing for the Dashboard to display. Troubleshooting this experiment usually means working backward through this chain to find where data stops flowing.

Table 9.1: Communication protocols in the IIoT pipeline.

#	Protocol	Role
1	Modbus	Communication between sensor and PLC
2	TCP/IP	Communication between PLC and WebAccess (SCADA)
3	ODBC	Communication between SCADA and the database
4	OPC UA	Alternative standard communication between PLC/equipment and SCADA



Figure 9.1: IIoT monitoring pipeline for this experiment.

Sensor Assignment

Each lab group is assigned a unique pair of parameters to monitor for this experiment. Pairing two signals per group, drawn from the kit's full 13-parameter OEE inventory (environmental, vibration, part-count/quality, availability, and operator/safety I/O), ensures each team independently validates its own signal path (PLC addressing, tag configuration, alarm thresholds, and dashboard binding) rather than inheriting settings tuned for a different group.

Refer to Table 9.2 for your group's assigned parameter pair, and to Table 9.3 for each parameter's PLC memory address.

Table 9.2: Sensor assignment by group (two parameters each).

Grp	Group focus	Parameters
G1	Environmental Climate Pair	Temperature & Humidity
G2	Dual-Axis Vibration Pair	Z-axis Vibration & X-axis Vibration
G3	Bearing Thermal-Vibration Pair	Z-axis Vibration & Vibration Temp
G4	Quality Defect Comparison	Good Part & Defect Part
G5	Good Yield Ratio Pair	Good Part & Total Output
G6	Scrap Rate Pair	Defect Part & Total Output
G7	Operator Control Console	Start Button & Stop Button
G8	Critical Safety Interlock	Stop Button & Emergency Stop
G9	Start Safety Interlock	Start Button & Emergency Stop
G10	Thermal Gradient Pair	Environmental Temp & Vibration Temp
G11	Production Rate & Uptime	Availability & Total Output
G12	Uptime & Quality Yield	Availability & Good Part
G13	Downtime & Scrap Correlation	Availability & Defect Part
G14	Position & Motion Sensing	Z-axis Vibration & Distance Sensor
G15	Part Sensing & Run Command	Distance Sensor & Start Button
G16	Humidity & Structural Dynamics	Humidity & X-axis Vibration
G17	Overheat Emergency Interlock	Vibration Temp & Emergency Stop
G18	Part Sensing & Tally Counter	Distance Sensor & Total Output

The 13 underlying parameters, their PLC memory addresses, and engineering notes are summarised in Table 9.3.

Table 9.3: S7-1200 OEE monitoring system — parameter inventory.

#	Parameter description	PLC memory address	Signal type	Engineering notes
1	Temperature (S15S-TH-MQ)	DBD3, 96	Analog	Environmental Temp (°C)
2	Humidity (S15S-TH-MQ)	DBD3, 92	Analog	Environmental Humidity (%RH)
3	Vibration (Z-axis RMS velocity)	DBD3, 4	Analog	Z-axis RMS velocity (mm/sec)
4	Vibration (X-axis RMS velocity)	DBD3, 20	Analog	X-axis RMS velocity (mm/sec)
5	Vibration (Temperature)	DBD3, 12	Analog	Sensor/Bearing Temp (°C)
6	Distance sensor	DBD3, 88	Analog	Object distance sensing (mm)
7	Start button	DB1, DBX0.1	Digital	Control panel Start command
8	Stop Button	DB1, DBX0.3	Digital	Control panel Stop command
9	Emergency stop	DB1, DBX0.0	Digital	Safety E-stop trigger
10	Availability	DBW304, 0	Metric/OEE	Equipment Availability status/pct
11	Good part	DBW8, 26	Counter	Accepted production tally
12	Defect part	DBW8, 28	Counter	Rejected production tally
13	Total output (Good + Defect)	DBW8, 30	Counter	Total manufactured count

Every group should create two IO tags in WebAccess/SCADA, one for each assigned parameter, so that your dashboard has two independent trends (or a trend and a status indicator, for a pair that mixes an analog and a discrete signal) to display.

Procedure

Activity 1 — Create the SCADA project and connect to the PLC

Objective: create a WebAccess/SCADA project and bring your assigned parameter pair in as live tags.

1. Open a web browser and navigate to `127.0.0.1` (or `localhost`) to reach the WebAccess/SCADA landing page. Select **Project Management** from the **Platform** dropdown and log in (default account `admin`, no password).



Figure 9.2: WebAccess/SCADA sign-in, with Project Management selected.

2. Create a new project: enter a **Project Name** and **Description**, set the **Project Node IP Address** (`127.0.0.1` for local), and keep the default **Primary TCP Port** (4592) and **HTTP Port** (80) unless instructed otherwise.

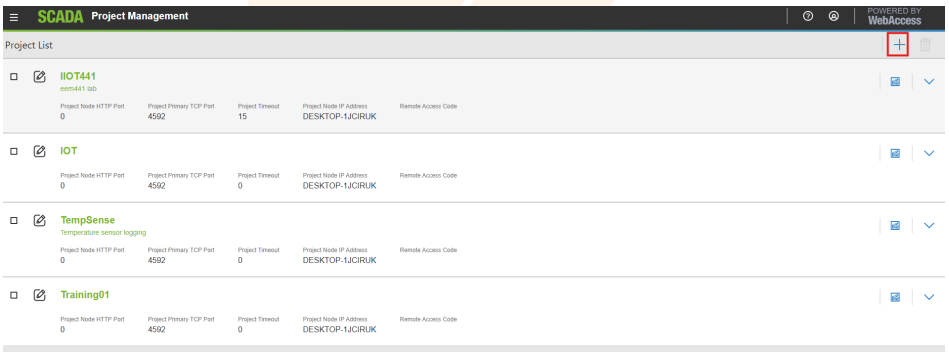


Figure 9.3: Create New Project form.

3. Create a **SCADA Node**: select your Project Node, click **+**, choose **New Node**, and fill in the Primary Node Name, TCP Ports, Node Timeout, and SCADA Node IP Address.



Figure 9.4: Create New SCADA Node form.

4. Add a **Communication Port**: choose **TCP/IP** as the interface, and configure the Scan Time (e.g. 1000 ms) and Auto-Recover Time.

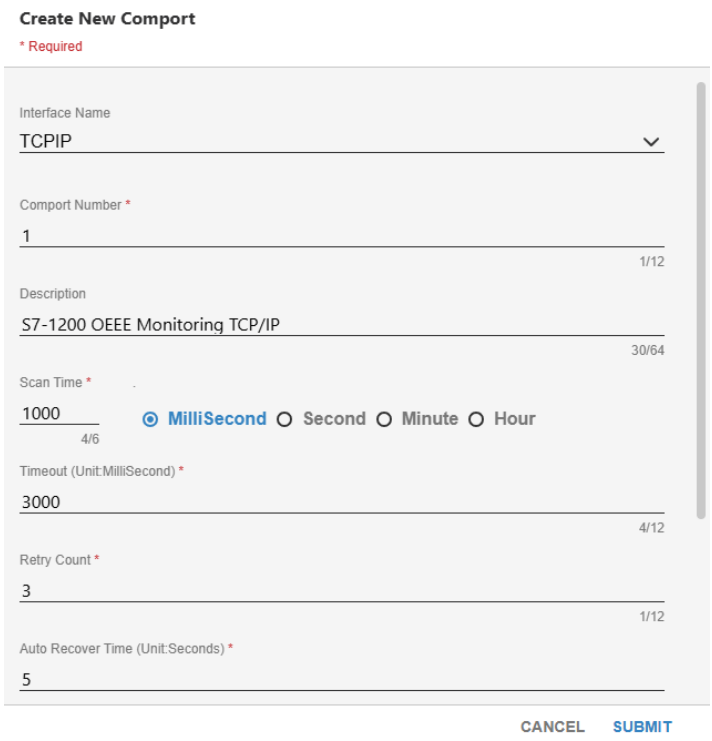


Figure 9.5: Communication port (Comport) settings.

5. Add a **Device** on that Comport: set Device Type to `SiemS7`, enter the PLC's IP address and port (`102`), and set the TSAP in hex for an S7-1200 (`01.01`).

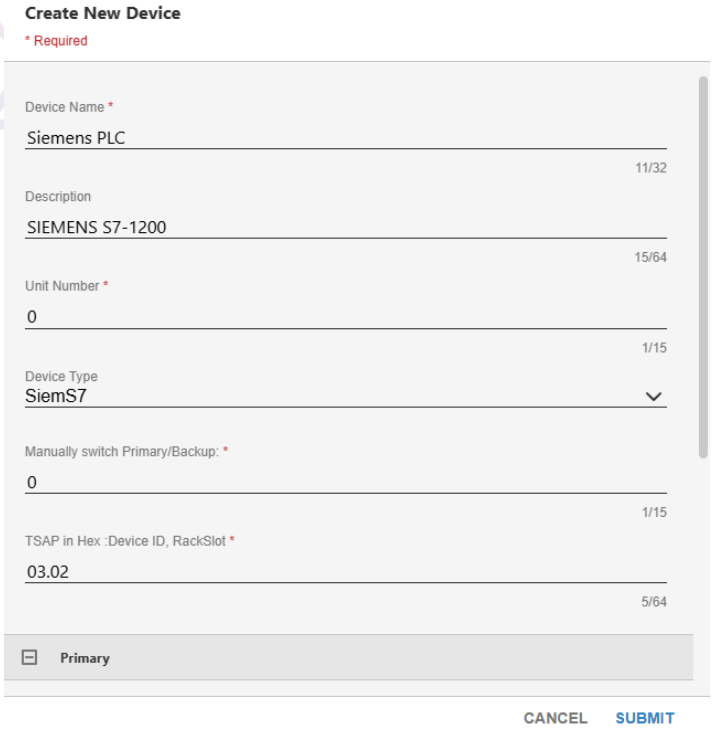


Figure 9.6: Add Device form (S7-1200).

6. Create two IO Tags, one for each parameter assigned to your group in Table 9.2 (e.g. `G1_Temp` and `G1_Humid` for Group 1). Set the Address, Data Type, and Conversion Code for each tag to match the PLC memory address given in Table 9.3.

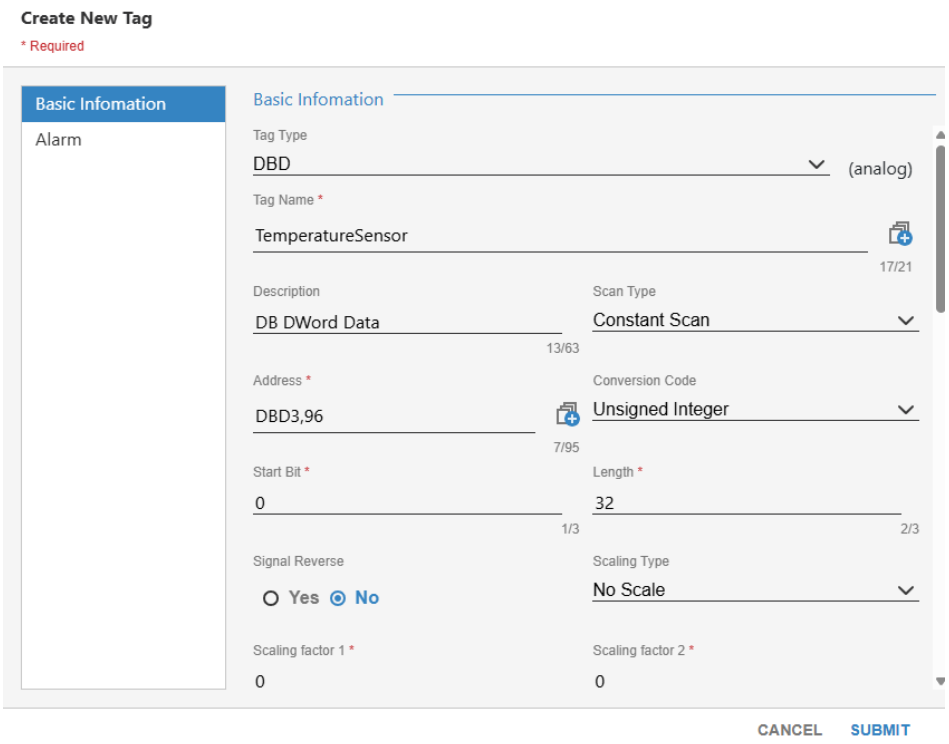


Figure 9.7: Insert IO Tag form (analog sensor example).

7. Start the SCADA Node's Kernel, open **ViewDAQ**, and confirm both tags are updating with live values that reflect what is actually happening on the machine.

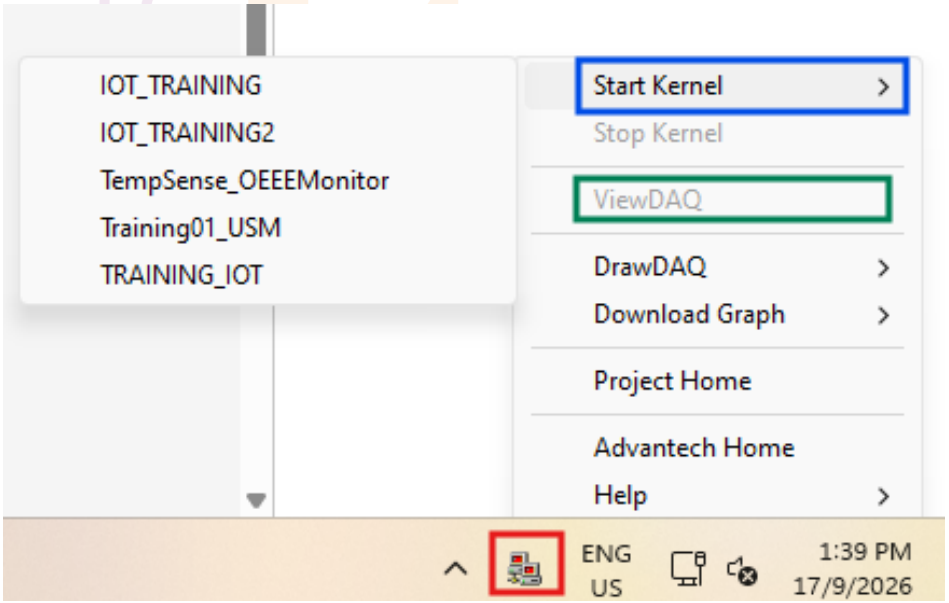


Figure 9.8: Kernel running, viewed in ViewDAQ.

Activity 2 — Configure an alarm

Objective: enable alarming on your assigned parameters, so threshold breaches are visible in both SCADA and, later, the dashboard.

1. For each **analog** tag in your pair (Signal type “Analog”, “Metric/OEE”, or “Counter” in Table 9.3), open its configuration and enable a non-zero alarm priority (High, High-High, Low, or Low-Low, as appropriate to that parameter).

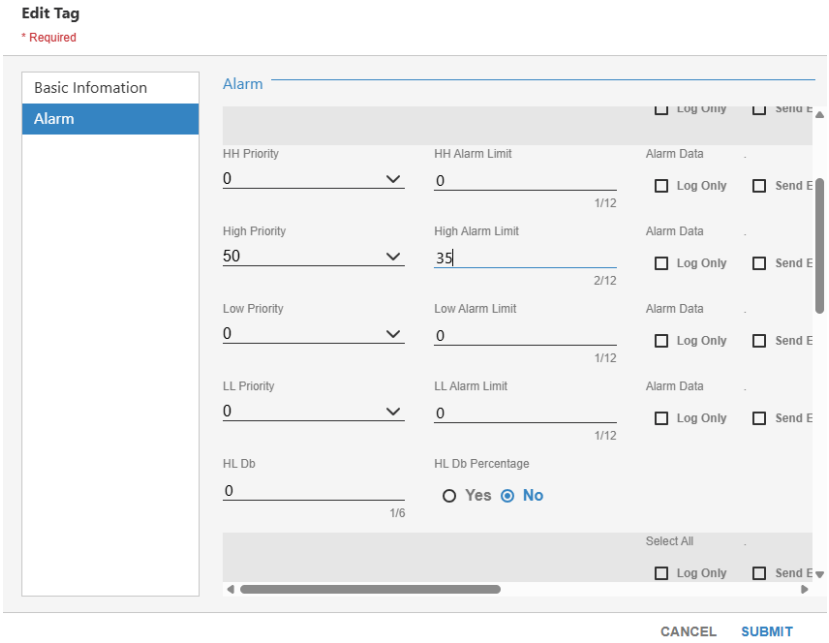


Figure 9.9: Alarm priority configuration.

2. For each **digital** tag in your pair (Signal type “Digital”), enable state alarming instead (e.g. alarm when Emergency Stop becomes active). If both of your assigned parameters are digital (Groups 7–9), configure state alarming on both.

3. In **ViewDAQ**, trigger a manual threshold breach on each tag (e.g. briefly block the sensor’s path, press a button, or induce vibration) and confirm the alarm appears in the SCADA **Alarm Summary**.

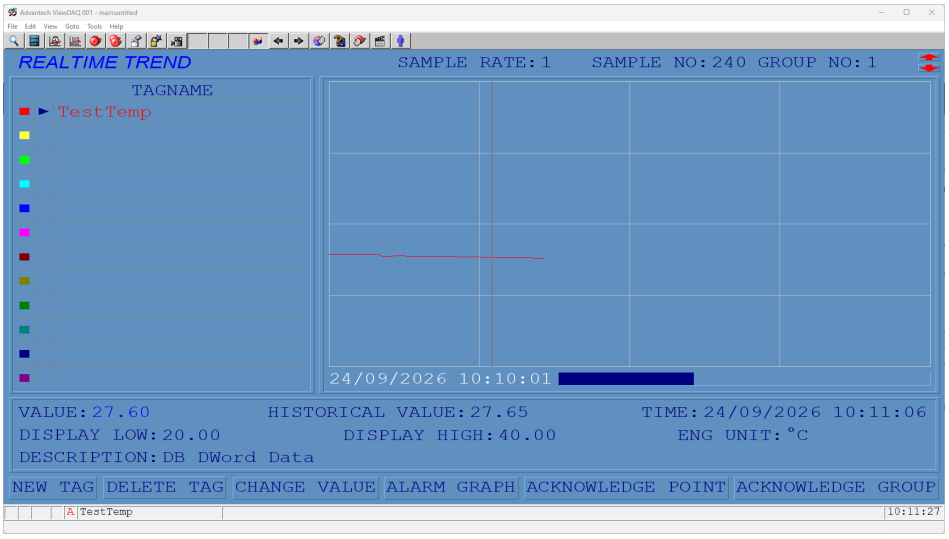


Figure 9.10: Alarm triggered, viewed on the real-time trend.

Activity 3 — Bridge SCADA to PostgreSQL via ODBC

Objective: connect WebAccess to a PostgreSQL database, and enable logging so your tag's history is written there.

1. Confirm with your instructor that PostgreSQL, the `psqlodbc` (x86) driver, and pgAdmin4 are already installed — this experiment assumes they exist; installing them from scratch is outside the scope of this lab.
2. In pgAdmin, create a connection to the PostgreSQL server (Host `127.0.0.1`, Port 5436, default credentials provided by your instructor), then create a new database for this experiment.

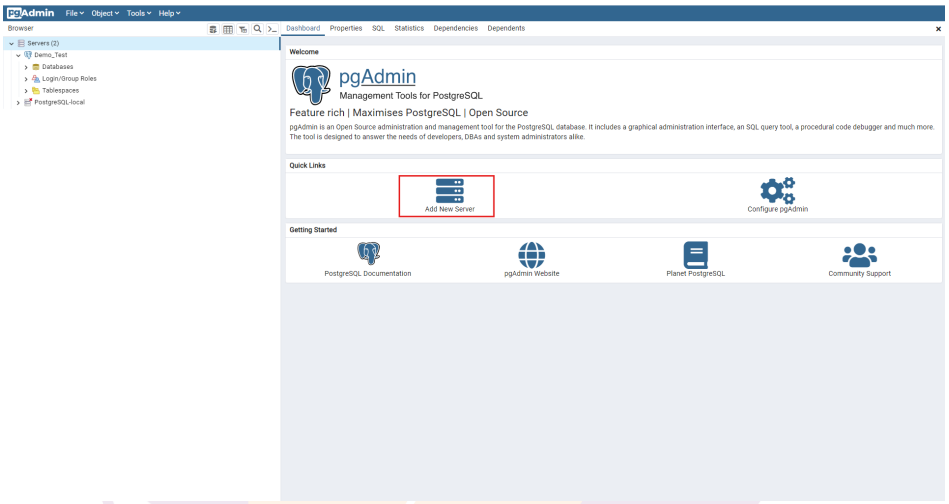


Figure 9.11: pgAdmin: registering the PostgreSQL server.

3. Open **ODBC Data Source Administrator (32-bit)**, switch to the **System DSN** tab, click **Add**, and select **PostgreSQL Unicode**. Fill in the Data Source name, Database, Server, Port, Username, and Password to match the database you just created.

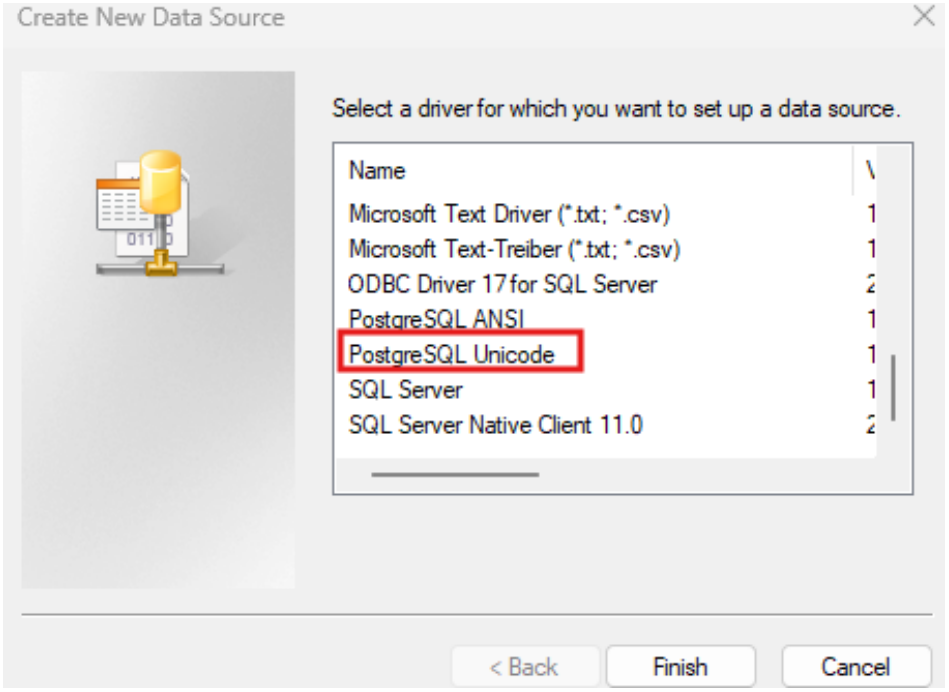


Figure 9.12: ODBC Data Source Administrator (32-bit).

4. Back in WebAccess, open the project's **Setting** menu → **ODBC data record source**, and insert a new ODBC entry: a numeric Log Data Source ID, the DSN name from the previous step, and matching login credentials. Test the connection.

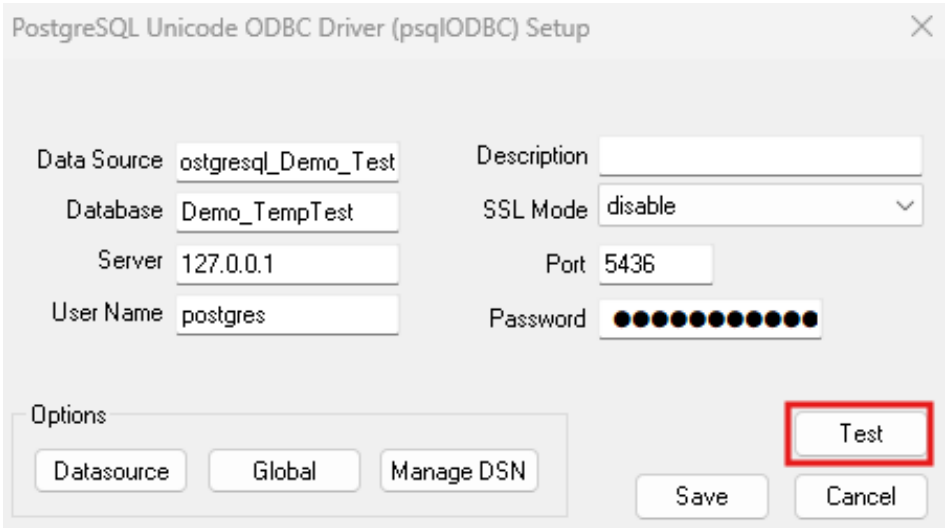


Figure 9.13: WebAccess ODBC data record source.

5. On your SCADA Node, enable **Data Log to ODBC** (**Edit Node** → **Data Log**). Then, on each tag you want logged, enable **Log Data** and set the **Data Log Interval** to `0` (record by deadband) to avoid missed samples.

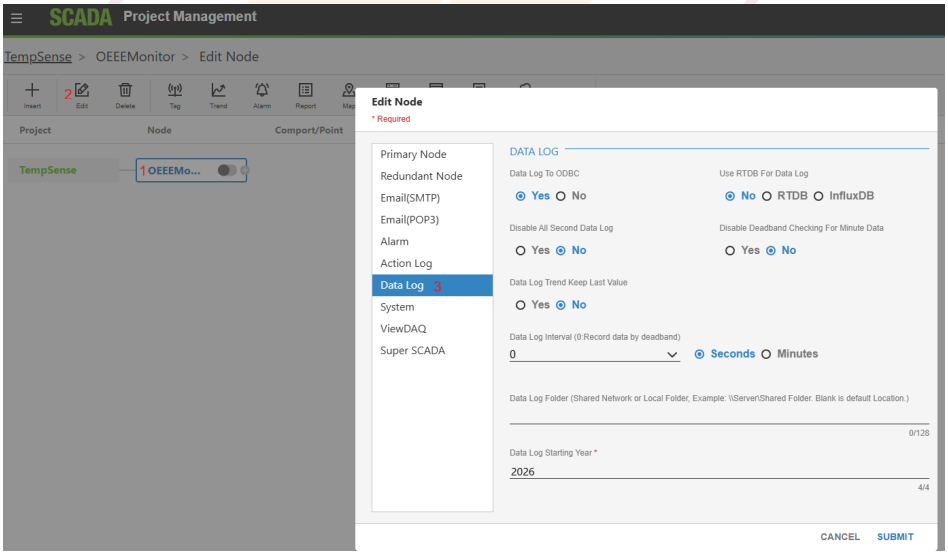


Figure 9.14: SCADA Node Edit Node: Data Log tab.

6. Download the project to the SCADA Node, then confirm in pgAdmin that rows are appearing in the corresponding log table (e.g. `bwanalogtable` for analog tags).

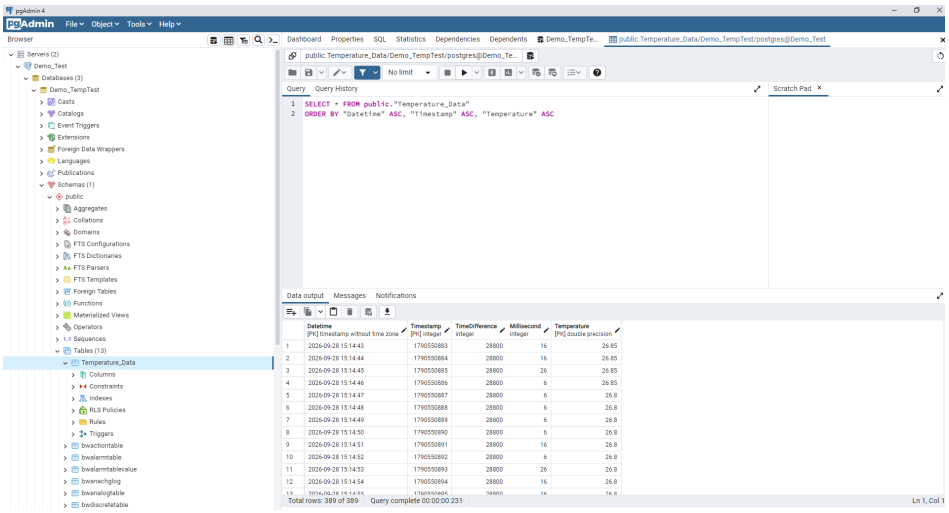


Figure 9.15: pgAdmin: logged rows in the analog log table.

Activity 4 — Build a dashboard

Objective: visualize your assigned parameter pair and its alarm state(s) on a WISE-IoTSuite/Dashboard panel.

1. From the WebAccess landing page, select **Dashboard** from the **Platform** drop-down instead of Project Management.
2. In the Dashboard app, open **Configuration → Data Sources → Add data source**, choose **PostgreSQL**, and enter the same connection details used for the ODBC bridge in Activity 3. Click **Save & Test** and confirm a successful connection.

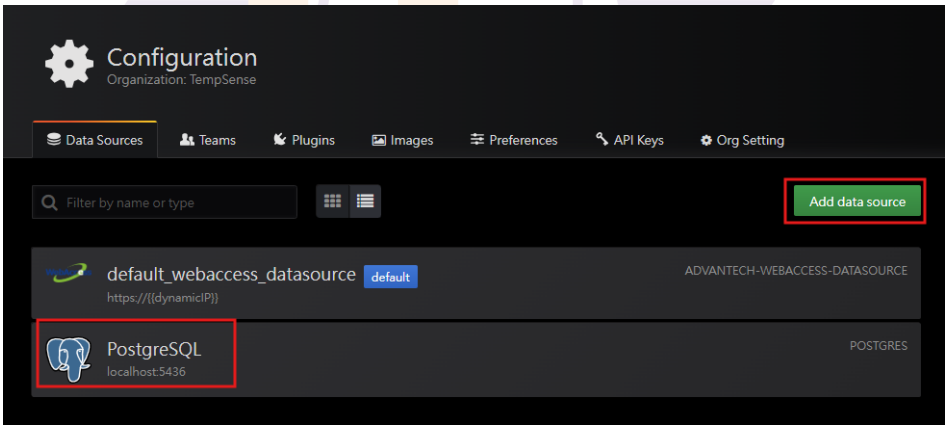


Figure 9.16: Dashboard: Add PostgreSQL data source.

3. Create a **new Dashboard**. Add at least three panels:
   - a **Singlestat** or **Graph** panel bound to each of your two assigned tags, and
   - an **Alert**-enabled panel (or an Alert list) showing the current state of your alarm(s) from Activity 2.

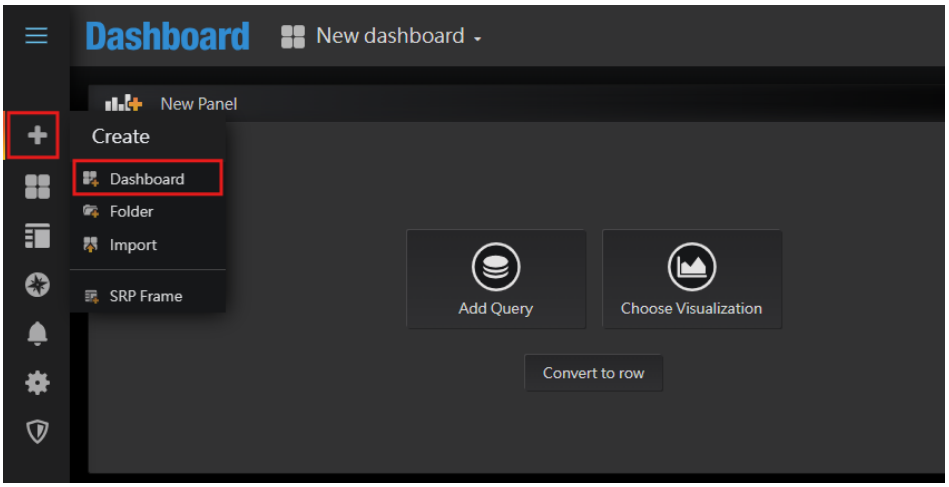


Figure 9.17: Dashboard panel query binding.

4. For each panel, select your PostgreSQL data source and the query/tag that corresponds to that assigned parameter.
5. Label your dashboard clearly with your group number, so it is identifiable in your submitted report.
6. With the machine running, confirm each panel updates live, and that triggering your alarm condition changes the alert panel's state.

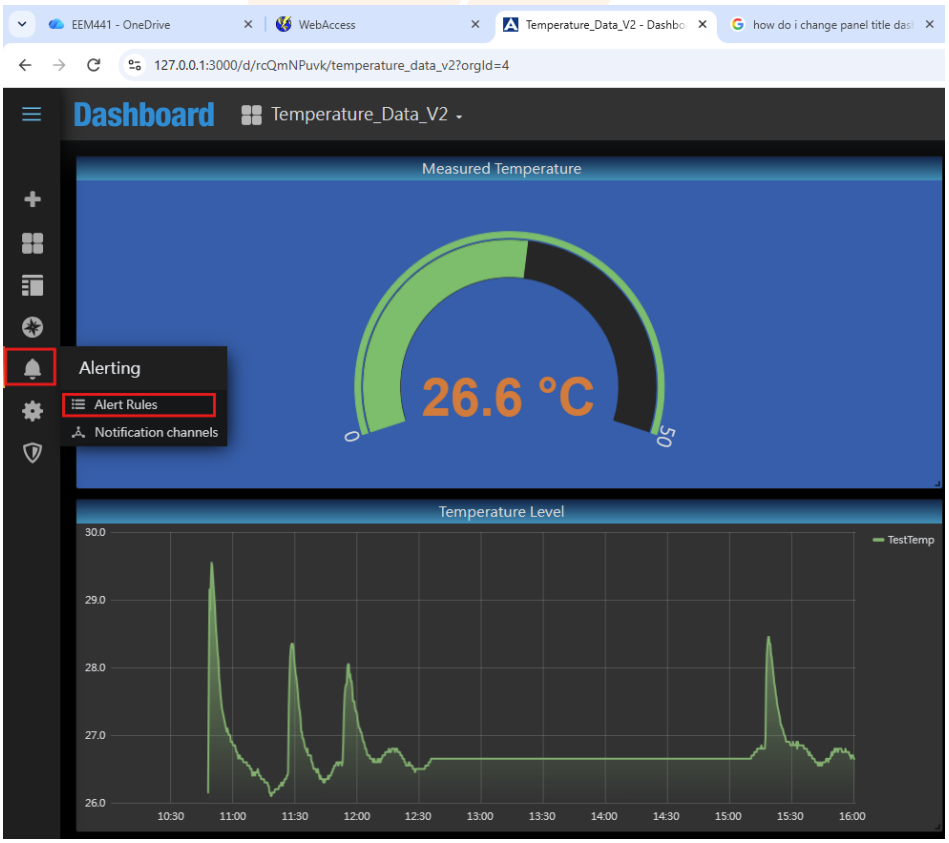


Figure 9.18: Finished dashboard example.

Report

1. Draw the complete data pipeline for your system (PLC → SCADA → database → dashboard), labelling the protocol used at each stage.
2. State your assigned sensor(s), their PLC DB address(es), and the WebAccess tag configuration you used to bring them into SCADA.
3. Describe the alarm(s) you configured, including threshold and priority for each, and show evidence (e.g. a screenshot of the Alarm Summary) that they trigger correctly.
4. Describe your ODBC bridge configuration (DSN name, database, and the WebAccess ODBC data record source settings), and explain what would go wrong if **Log Data** were left disabled on one of your tags.
5. Present a screenshot of your finished dashboard, and explain why you chose the panel types you used for your two parameters and their alarms.
6. Describe one problem you encountered in this experiment (in any layer of the pipeline) and how you diagnosed and resolved it.

Deliverables

1. **System configuration summary:** your PLC, SCADA, and ODBC settings, including the pipeline diagram and tag mapping from the Report questions.
2. **Dashboard screenshot(s):** your finished dashboard, clearly labelled with your group number.
3. **Short demonstration:** a live walkthrough (or short video) showing your dashboard updating in response to the physical sensor, and the alarm triggering correctly.
4. **Video:** a video of your work, showing your assigned parameters updating live on the dashboard and your alarm(s) triggering and clearing.

References

- MTA SkillLab Sdn Bhd, *i-Factory IIoT Training Manual*, Module 2 (IIoT Hardware & Software), Module 3 (Project Creation in SCADA), and Module 4 (Dashboard Viewing and Analysis).
- Advantech WebAccess/SCADA product documentation.
- PostgreSQL ODBC driver (`psqlodbc`): https://www.postgresql.org/ftp/odbc/versions/msi/

Appendix: Troubleshooting

This experiment's pipeline has four layers (Table 9.1), and almost every symptom traces back to exactly one of them. Work backward from where the data visibly stops — if the dashboard panel is blank, check whether the database has rows; if not, check whether SCADA shows a live value; if not, check the PLC connection.

Table 9.4: Common symptoms and their likely cause.

Symptom	Likely layer	What to check
Tag shows no value, or a constant 0, in ViewDAQ	PLC ↔ SCADA	PLC powered and reachable (ping); Comport/Device IP, port, and TSAP correct; PLC CPU in RUN mode, not STOP
Tag updates in ViewDAQ but PostgreSQL table is empty	SCADA ↔ Database	**Log Data** enabled on the tag; **Data Log to ODBC** enabled on the SCADA Node; Data Log Interval set to `0`
ODBC **Save & Test** fails (SCADA or Dashboard side)	Database connection	PostgreSQL service running; DSN name, Server, Port (5436), Username, and Password all match exactly
Dashboard panel shows "No data" despite a working data source	Dashboard query	Correct table/tag selected in the panel's Query settings; time range not excluding all logged rows
Alarm never appears in the Alarm Summary	SCADA alarm configuration	Alarm priority is non-zero; threshold value is actually crossed by the live signal, not just approached

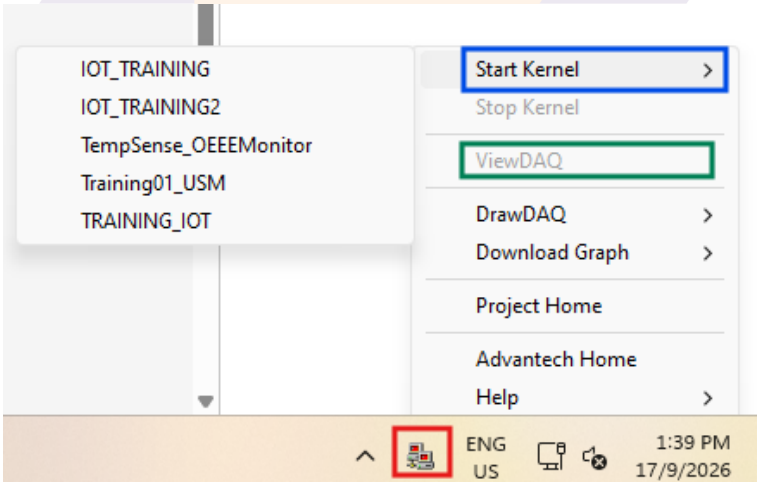


Figure 9.19: Kernel/ViewDAQ status — confirm the Kernel is actually running before suspecting anything further down the pipeline.

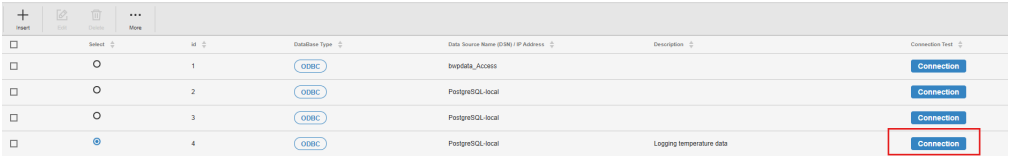


Figure 9.20: ODBC connection test.

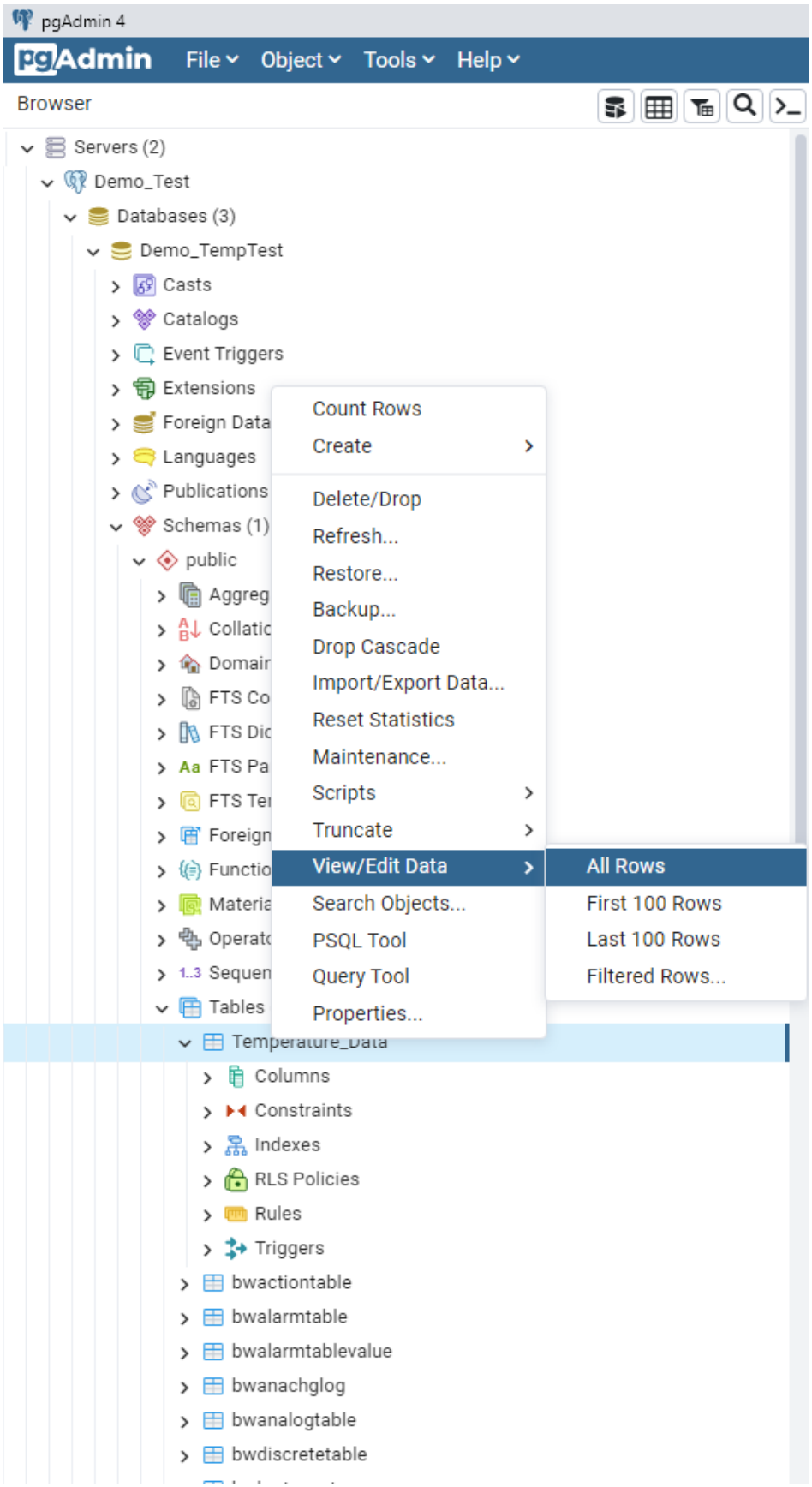


Figure 9.21: Viewing/editing log table data in pgAdmin — an empty result here points back to the SCADA-to-database layer.